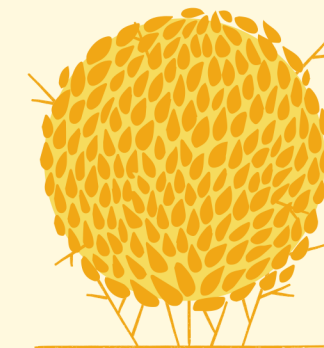
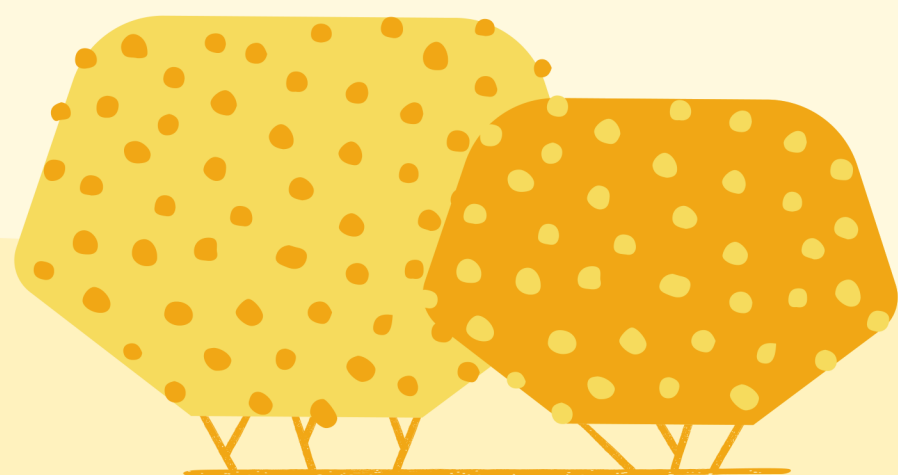




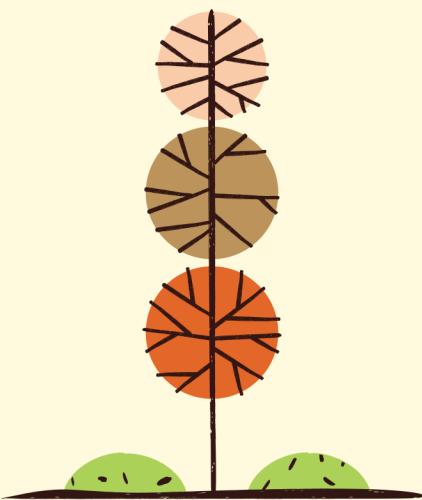
MANAGE SESSIONS WITH REFRESH TOKEN



Backend Master Class #37



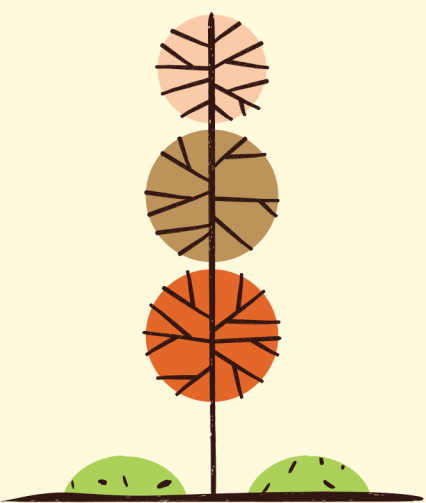
Advanced Backend Topics



Advanced Backend Topics



How to manage
user session?



Why PASETO is better than JWT?

Backend Master Class
Lecture 19

19

Why PASETO is better than JWT?

Backend Master Class
Lecture 19

19

Create & verify JWT & PASETO in Golang

Backend Master Class
Lecture 20

20



Why PASETO is better than JWT?

Backend Master Class
Lecture 19

19

Create & verify JWT & PASETO in Golang

Backend Master Class
Lecture 20

20



Backend Master Class
LECTURE 21

Implement login API with PASETO & JWT



21

Why PASETO is better than JWT?

Backend Master Class
Lecture 19

19

Create & verify JWT & PASETO in Golang

Backend Master Class
Lecture 20

20



Backend Master Class
LECTURE 21

Implement login API with PASETO & JWT



21

22

HOW TO IMPLEMENT AUTHENTICATION MIDDLEWARE

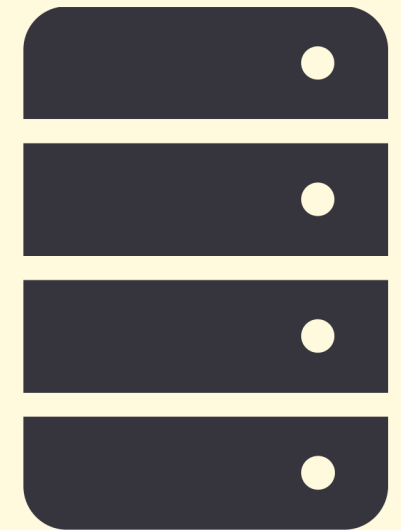
IN GOLANG - GIN FRAMEWORK



Token-based Authentication



Client



Server

1. POST /users/login

{username, password}

200 OK

{access_token: JWT, PASETO, ...}

Sign token

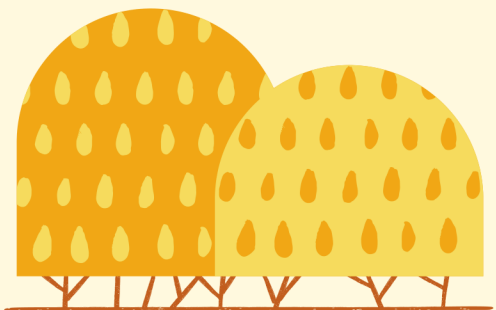
2. GET /accounts

Authorization: Bearer <access_token>

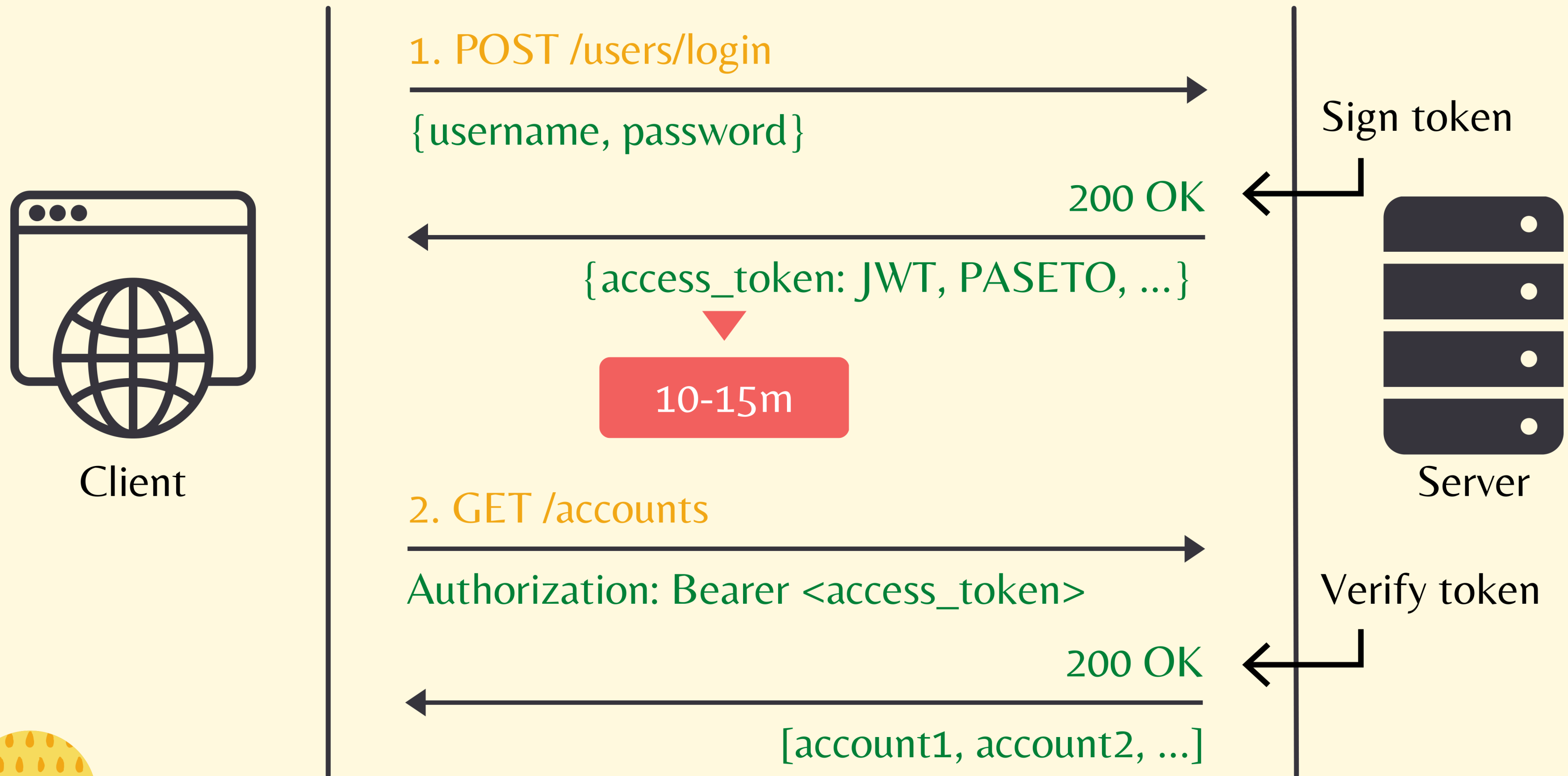
200 OK

[account1, account2, ...]

Verify token



Token-based Authentication



Token expiration



Client

1. POST /users/login

{username, password}

200 OK

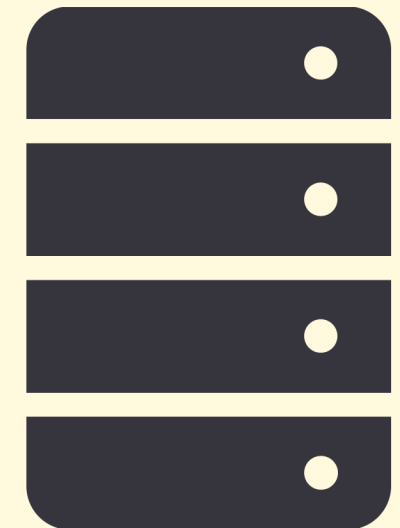
{access_token: JWT, PASETO, ...}

10-15m

2. GET /accounts

Authorization: Bearer <access_token>

Sign token



Server

Verify token



Token expiration



Client

1. POST /users/login

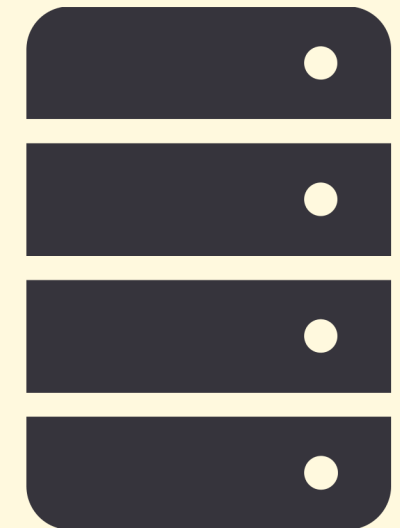
{username, password}

200 OK

{access_token: JWT, PASETO, ...}

10-15m

Sign token



Server

2. GET /accounts

Authorization: Bearer <access_token>

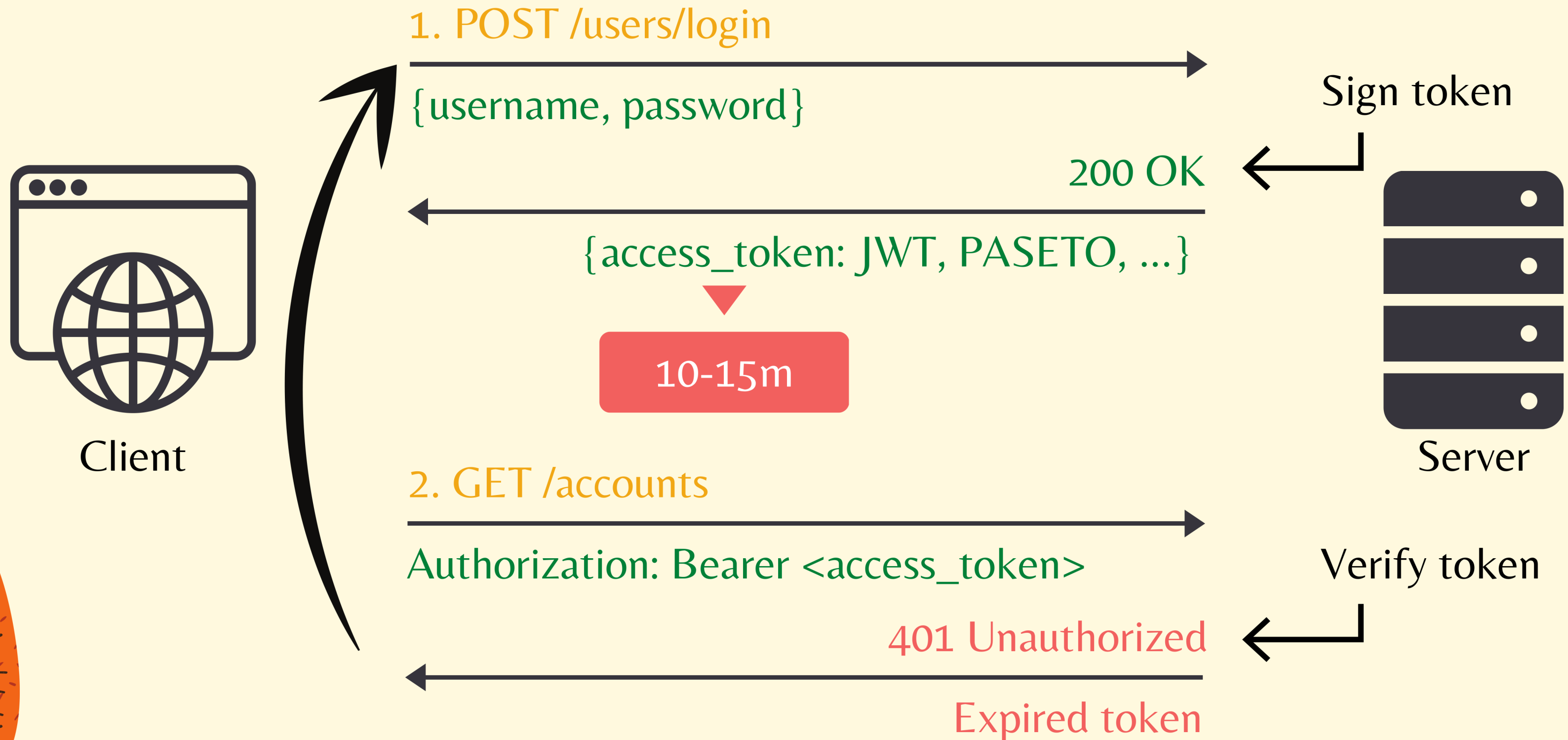
401 Unauthorized

Expired token

Verify token



Token expiration



Session management

1. POST /users/login

{username, password}

200 OK

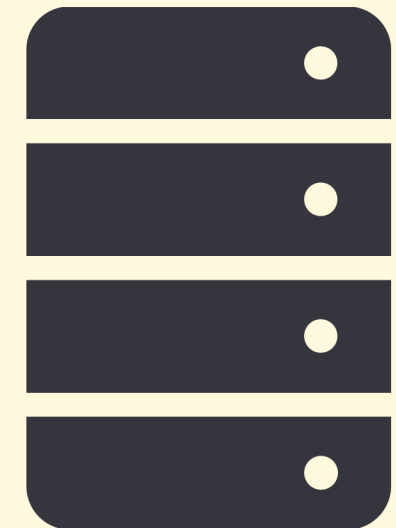
{access_token + refresh_token}

10-15m

Sign token



Client



Server



Session management

1. POST /users/login

{username, password}

200 OK

{access_token + refresh_token}

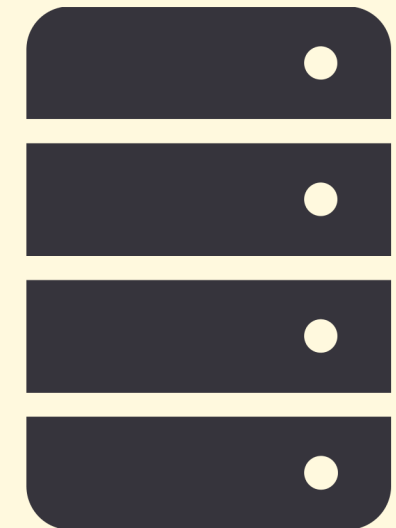
10-15m

1-7 days

Sign token



Client



Server



Session management



Client

1. POST /users/login

{username, password}

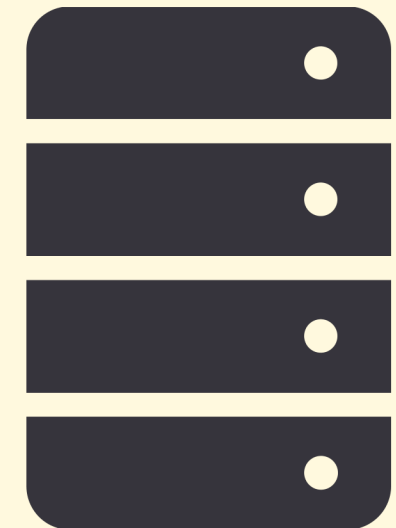
200 OK

{access_token + refresh_token}

10-15m

1-7 days

Sign token



Server

2. POST /tokens/renew_access

{refresh_token}



Session management



Client

1. POST /users/login

{username, password}

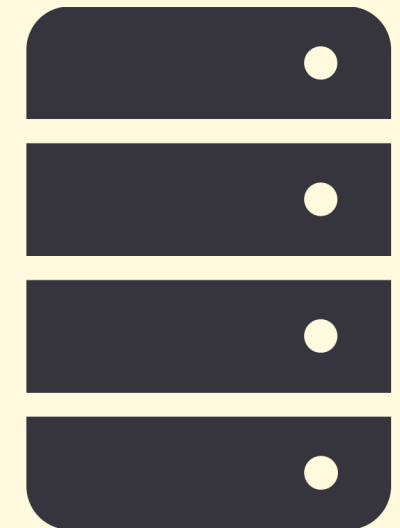
200 OK

{access_token + refresh_token}

10-15m

1-7 days

Sign token



Server

2. POST /tokens/renew_access

{refresh_token}

is_blocked?

Verify token



Session management



Client

1. POST /users/login

{username, password}

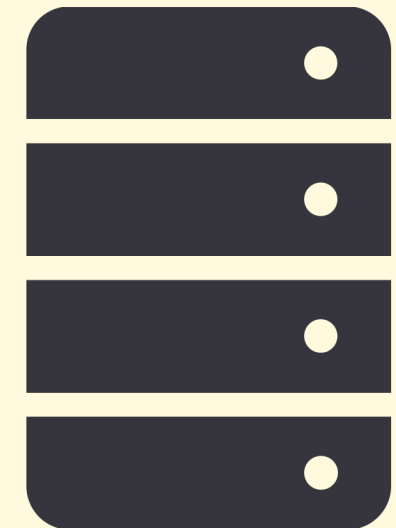
200 OK

{access_token + refresh_token}

10-15m

1-7 days

Sign token



Server

2. POST /tokens/renew_access

{refresh_token}

200 OK

{access_token}

is_blocked?

Verify token

